

Compiler Project

This project is a simple Compiler Construction tool for a Tiny Programming Language.
It demonstrates the first phase of a compiler, which is Lexical Analysis, by transforming a given language specification into formal models and a working GUI tool.

Name	Section
Zeyad Yasser Salah	3
Seif-Eldeen Taha	3
Seif Mujahid	3
Zeyad Magdy	3
Ibrahim Mokther	1

Under Supervision: Eng Crestien

1. Language Tokens

a. Reserved Words

- if
 - else
 - then
 - end
 - repeat
 - until
 - read
 - write
-

b. Operators

- Arithmetic Operators: +, -, *, /
 - Relational Operators: <, >, <=, >=
 - Assignment Operator: :=
-

c. Symbols

- ;
 - (
 -)
-

d. Identifiers

- Definition:
Identifier $\rightarrow$ Letter (Letter | Digit)*
-

e. Numbers

- Definition:
Number $\rightarrow$ digit+
or
digit (digit)*

f. Comments

- Definition:
 $\backslash[^\backslash]^*\backslash$

2. Language Definition using Regular Expressions

This section defines the lexical structure of the language using regular expressions. It describes how different tokens such as identifiers, numbers, keywords, operators, symbols, and comments are formed.

Identifier

Identifier $\rightarrow$ letter (letter | digit)*

- Starts with a letter.
 - Can be followed by zero or more letters or digits.
-

Number

Number $\rightarrow$ digit+

- Consists of one or more digits.
 - Represents integer values.
-

Keyword

Keyword $\rightarrow$ if | then | else | end | repeat | until | read | write

- Reserved words with predefined meanings in the language.
- Cannot be used as identifiers.

Operator

Operator $\rightarrow + \mid - \mid * \mid / \mid = \mid < \mid :=$

- Includes arithmetic, relational, and assignment operators.

Symbol

Symbol $\rightarrow ; \mid (\mid) \mid ,$

- Special characters used for structuring the program.

Assignment

Assignment $\rightarrow :=$

- Used to assign values to variables.

Comment

Comment $\rightarrow \backslash \{ [^] * \}$

Explanation of the Comment Regular Expression

The comment is defined as:

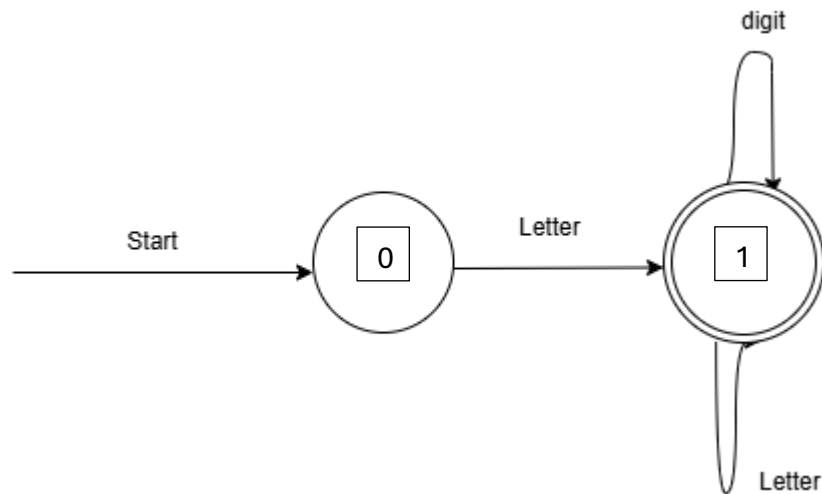
$\backslash \{ [^] * \}$

Breakdown:

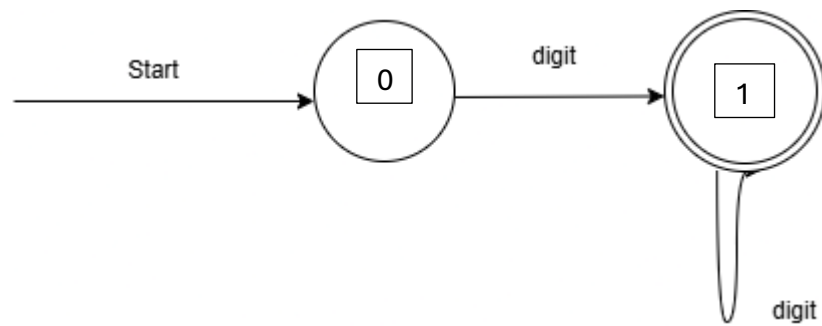
- $\backslash \{ \rightarrow$ Matches the opening curly brace {
- $[^] \rightarrow$ Matches any character except }

- * → Means "zero or more repetitions"
- \} → Matches the closing curly brace }

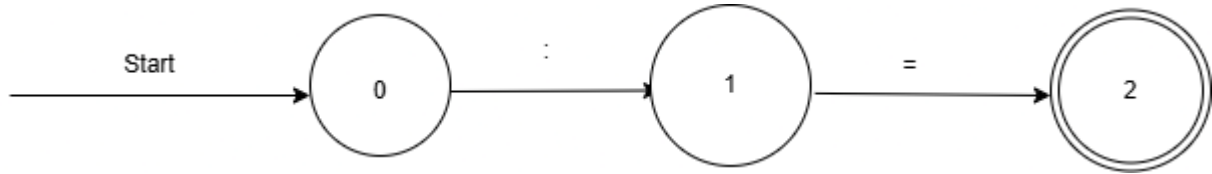
NFA For Identifier:



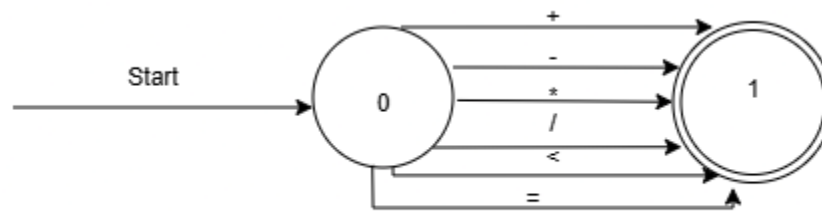
NFA For Number:



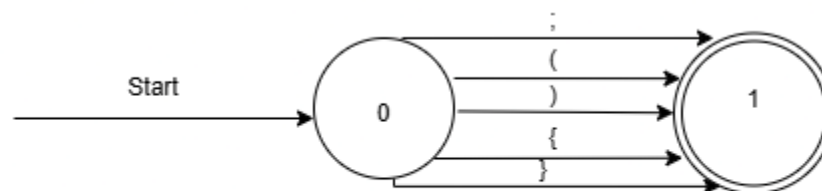
NFA For Assign Operator:



Nfa For Operators :



NFA for Symbols :



NFA For Comment :

